

JPEG 与 JPEG 2000 压缩原理

详解

独立知识速查文件：不依赖、不引用本仓库任何其他文件，也不被其他文件引用。用途：随时查阅 JPEG / JPEG 2000 的压缩原理、公式、参数表与实现细节。

目录

第一部分 JPEG - 1. 标准家族与编号 - 2. 编码总流程 - 3. 色彩空间转换与子采样 - 4. 分块与 Level Shift - 5. DCT (离散余弦变换) - 6. 量化 (唯一有损步骤) - 7. ZigZag 扫描 - 8. 熵编码前的处理 - 9. 霍夫曼编码 - 10. 算术编码 (可选模式) - 11. 文件结构 (标记段) - 12. 渐进式 JPEG (Progressive) - 13. 无损 JPEG (SOF3) - 14. 重启间隔 (Restart Interval) - 15. 典型伪影 (Artifacts) - 16. 解码流程

第二部分 JPEG 2000 - 1. 设计目标 - 2. 编码总流程 - 3. 分量变换 - 4. 分块 (Tile) - 5. 小波变换 (DWT) - 6. 量化 - 7. EBCOT Tier-1: 位平面编码 - 8. EBCOT Tier-2: 分层打包 - 9. 三重渐进性 - 10. 感兴趣区域 (ROI) - 11. 文件格式 - 12. JPEG 2000 家族成员

第三部分 对比与选型 - 1. JPEG vs JPEG 2000 全面对比 - 2. 应用场景 - 3. 为什么 JPEG 2000 没有取代 JPEG - 4. 常见误区

附录 速查表 - A. JPEG 标记段速查 - B. JPEG 2000 标记段速查 - C. 关键公式速查 - D. 参数默认值速查 - E. 术语中英对照 - F. JPEG 2000 量化参数速查

第一部分 JPEG

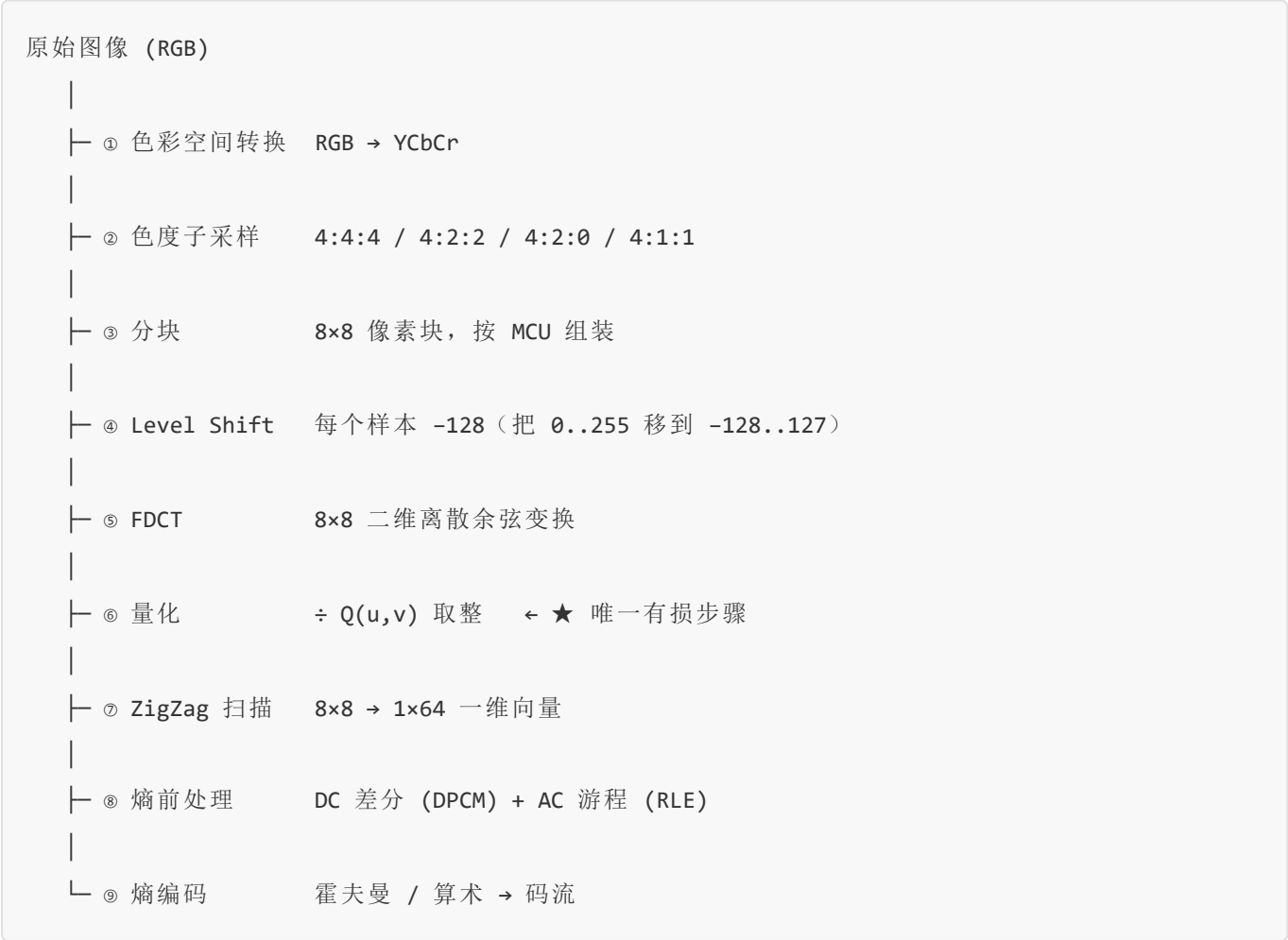
1. 标准家族与编号

标准号	俗称	说明
ISO/IEC 10918-1	JPEG (基线)	有损 DCT 编码, 最广泛使用
ISO/IEC 10918-2	一致性测试	定义测试方法
ISO/IEC 10918-3	JPEG 扩展	SPIFF 文件格式、分层、16-bit 等
ISO/IEC 10918-4	注册	注册标记与标识符
ISO/IEC 14495	JPEG-LS	无损/近无损, LOCO-I 算法, 医学影像常用
ISO/IEC 15444	JPEG 2000	小波变换, 见第二部分
ISO/IEC 29199-2	JPEG XR	原 HD Photo / Windows Media Photo
ISO/IEC 18477	JPEG XT	在 JPEG 上叠加 HDR / 无损 / Alpha
ISO/IEC 21122	JPEG XS	低延迟轻量压缩, 用于视频链路
ISO/IEC 18181	JPEG XL	新一代通用格式, 含无损重压缩 JPEG

1. 四种编码模式 (由 SOF 标记区分)

模式	SOF 标记	说明
基线顺序 (Baseline Sequential)	SOF0 (0xFFC0)	8-bit、霍夫曼编码, 必须支持
扩展顺序 (Extended Sequential)	SOF1 (0xFFC1)	12-bit、算术编码
渐进 (Progressive)	SOF2 (0xFFC2)	多次扫描, 逐步清晰
无损 (Lossless)	SOF3 (0xFFC3)	预测编码, 无 DCT
差分/分层	SOF5 – SOF15	少见

2. 编码总流程



解码是严格逆过程：熵解码 → 反 ZigZag → 反量化 → IDCT → +128 → 上采样 → YCbCr→RGB。

关键认知：JPEG 的"有损"**只来自第 ⑥ 步量化**。其余步骤 (DCT、ZigZag、霍夫曼) 全部可逆。所以"提高 JPEG 质量"本质就是"减小量化步长"。

3. 色彩空间转换与子采样

1. RGB → YCbCr (JFIF / Rec.601)

$$Y = 0.299R + 0.587G + 0.114B$$

$$Cb = 128 - 0.168736R - 0.331264G + 0.5B$$

$$Cr = 128 + 0.5R - 0.418688G - 0.081312B$$

逆变换：

$$R = Y + 1.402(Cr - 128)$$

$$G = Y - 0.344136(Cb - 128) - 0.714136(Cr - 128)$$

$$B = Y + 1.772(Cb - 128)$$

+128 是 8-bit 无符号偏移，把有符号色差搬到 0..255。

2. 为什么要转 YCbCr

人眼对亮度细节敏感、对色度细节迟钝。转成 YCbCr 后可以： 1. 对 Cb/Cr 降分辨率（子采样）而不易察觉； 2. 对 Cb/Cr 用更粗的量化表（标准色度量化表比亮度表粗得多）。

3. 子采样格式

格式	色度分辨率	8-bit 位深	说明
4:4:4	1×1	24 bit/px	不采样
4:2:2	1/2×1	16 bit/px	水平减半
4:2:0	1/2×1/2	12 bit/px	横竖各减半（JPEG 最常用）
4:1:1	1/4×1	12 bit/px	水平取 1/4

JPEG 默认 4:2:0 (Y 的 h1:v1 = 2:1，Cb/Cr 为 1:1)。

4. MCU（最小编码单元）

采样格式	MCU 组成	块数
4:4:4	1 Y + 1 Cb + 1 Cr	3
4:2:2	2 Y + 1 Cb + 1 Cr	4
4:2:0	4 Y + 1 Cb + 1 Cr	6

MCU 是熵编码的最小交织单位，也是重启间隔的计数单位。

4. 分块与 Level Shift

1. 8×8 分块

图像被切成互不重叠的 8×8 块。边缘不足 8 像素时**复制边缘像素填充** (padding) 。

分块是 JPEG 块效应 (blocking artifact) 的根源：块与块之间独立量化，边界处不连续。

2. Level Shift

$$f(x, y) = \text{pixel}(x, y) - 128$$

把无符号 0..255 变成有符号 -128..127，使 DCT 输入的均值接近 0，**降低 DC 系数幅度**，提高压缩率。

5. DCT（离散余弦变换）

1. 正变换 FDCT

$$F(u, v) = \frac{1}{4} C(u) C(v) \sum_{x=0}^7 \sum_{y=0}^7 f(x, y) \cos \frac{(2x+1)u\pi}{16} \cos \frac{(2y+1)v\pi}{16}$$

其中

$$C(k) = \begin{cases} \frac{1}{\sqrt{2}} & k = 0 \\ 1 & k > 0 \end{cases}$$

2. 逆变换 IDCT

$$f(x, y) = \frac{1}{4} \sum_{u=0}^7 \sum_{v=0}^7 C(u) C(v) F(u, v) \cos \frac{(2x+1)u\pi}{16} \cos \frac{(2y+1)v\pi}{16}$$

3. 系数含义

- $F(0, 0)$ = **DC 系数** = 64 个样本的平均值 $\times 8$ （因 $C(0)C(0)/4 = 1/8$ ，乘 64 得 8）
- $F(u, v)$ （ u, v 不全为 0）= **AC 系数**，代表不同空间频率的分量

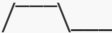
8×8 块的 DCT 系数布局（频率从低到高）



4. 为什么用 DCT 而不是 DFT

特性	说明
能量集中	自然图像高度相关，DCT 把能量压缩到少数低频系数（接近 KLT 的最优性能）
无边界不连续	DCT 隐含偶对称延拓，不像 DFT 隐含周期延拓会在块边界产生跳变
可分离	二维 DCT = 先对行做一维 DCT，再对列做一维 DCT，复杂度从 $O(N^4)$ 降到 $O(N^3)$
实数运算	系数全为实数，无需复数运算
快速算法	类似 FFT 的快速 DCT，8 点 DCT 只需约 29 次乘法

5. 一维 DCT 基函数 (8 点)

$u=0$ (DC)		常数
$u=1$		半个余弦波
$u=2$		一个余弦波
$u=3$		一个半余弦波
$u=4$		两个余弦波
$u=5..7$		频率更高，幅度更小

6. 量化（唯一有损步骤）

1. 公式

量化:

$$F_q(u, v) = \text{round}\left(\frac{F(u, v)}{Q(u, v)}\right)$$

反量化:

$$\hat{F}(u, v) = F_q(u, v) \cdot Q(u, v)$$

误差上界: $|F - \hat{F}| \leq Q(u, v)/2$

2. 标准亮度量化表 (JPEG Annex K)

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

3. 标准色度量化表

17	18	24	47	99	99	99	99
18	21	26	66	99	99	99	99
24	26	56	99	99	99	99	99
47	66	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99

观察：左上角（低频）值小 → 保留精细；右下角（高频）值大 → 大量归零。色度表整体比亮度表粗（人眼对色度不敏感）。

4. 质量因子 Q 缩放

给定质量 $Q \in [1, 100]$ ，缩放系数 S ：

$$S = \begin{cases} \frac{5000}{Q} & Q < 50 \\ 200 - 2Q & Q \geq 50 \end{cases}$$

$$Q_{\text{scaled}}(u, v) = \text{clamp}\left(\text{round}\left(\frac{S \cdot Q_{\text{base}}(u, v)}{100}\right), 1, 255\right)$$

Q	S	效果
100	0	所有量化步长 → 1（近似无损，但仍有舍入）
90	20	高质量
75	50	标准默认，视觉良好
50	100	使用原始标准表
25	200	明显劣化
10	500	严重块效应

注意： `Q=100` 不等于无损。DCT/IDCT 的浮点舍入 + 色度子采样仍会引入误差。真正的无损要用 `SOF3` 或 PNG / JPEG-LS。

7. ZigZag 扫描

把 8×8 矩阵按"之"字形读成一维 64 元素向量，使**低频在前、高频在后**，便于后续游程编码把尾部连续的 0 压缩掉。

ZigZag 读取顺序（数字 = 读取次序）

```
0  1  5  6 14 15 27 28
2  4  7 13 16 26 29 42
3  8 12 17 25 30 41 43
9 11 18 24 31 40 44 53
10 19 23 32 39 45 52 54
20 22 33 38 46 51 55 60
21 34 37 47 50 56 59 61
35 36 48 49 57 58 62 63
```

一维向量： [DC, AC1, AC2, ..., AC63]

8. 熵编码前的处理

1. DC 系数：差分编码（DPCM）

相邻 8×8 块的 DC 系数高度相关，所以不直接编码 DC 值，而是编码与前一块 DC 的差值：

$$\text{Diff} = DC_i - DC_{i-1}$$

- 每个分量（Y、Cb、Cr）各自维护一个 DC 预测器；
- 每个重启间隔（restart interval）开始时预测器重置为 0。

2. AC 系数：游程编码（RLE）

AC 系数经量化后大量为 0，用 (RUN, SIZE) 对表示：

- RUN = 前面连续 0 的个数（0–15）
- SIZE = 非零系数所需的位数（0–10）

两个特殊符号：

符号	名称	含义
(0, 0)	EOB (End of Block)	后面全是 0，块结束
(15, 0)	ZRL (Zero Run Length)	表示 16 个连续的 0

3. 数值编码：SIZE + 附加位

JPEG 用"类别 + 幅值"表示整数：

SIZE	可表示范围	附加位
0	0	无
1	-1, 1	1 bit
2	-3..-2, 2..3	2 bit
3	-7..-4, 4..7	3 bit
...	...	...
11	-2047..-1024, 1024..2047	11 bit

负数编码规则：对值 v ，若 $v < 0$ ，则编码为 $v + 2^{\text{SIZE}} - 1$ （即按位取反的低位形式）。

例： $v = -5$ ， $\text{SIZE} = 3$ ， 编码为 $-5 + 8 - 1 = 2 \rightarrow$ 二进制 `010`。

9. 霍夫曼编码

1. 编码对象

符号类型	组成
DC 符号	SIZE (1 个霍夫曼码) + 附加位 (SIZE 位)
AC 符号	(RUN, SIZE) (1 个霍夫曼码) + 附加位 (SIZE 位)

2. 标准霍夫曼表 (Annex K)

JPEG 规定了 4 张默认表：DC 亮度、DC 色度、AC 亮度、AC 色度。

DC 亮度表：

```
BITS      = [0, 1, 5, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0]
HUFFVAL   = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
```

即：码长 2 有 1 个符号 (0)，码长 3 有 5 个符号 (1-5)，码长 4-9 各 1 个。

AC 亮度表：共 162 个符号，覆盖所有常见 (RUN, SIZE) 组合。

3. 优化霍夫曼表

编码器可先统计图像的实际符号频率，用霍夫曼算法生成最优表，写入 DHT 段。代价是必须把表存进文件 (约 400 字节开销)。

4. 字节填充 (Byte Stuffing)

熵编码码流中若出现 0xFF，后面必须补一个 0x00，避免与标记段冲突。

- 编码：遇到 0xFF → 输出 0xFF 0x00
- 解码：读到 0xFF 后看下一字节，若是 0x00 则还原为 0xFF；若是其他则说明是标记

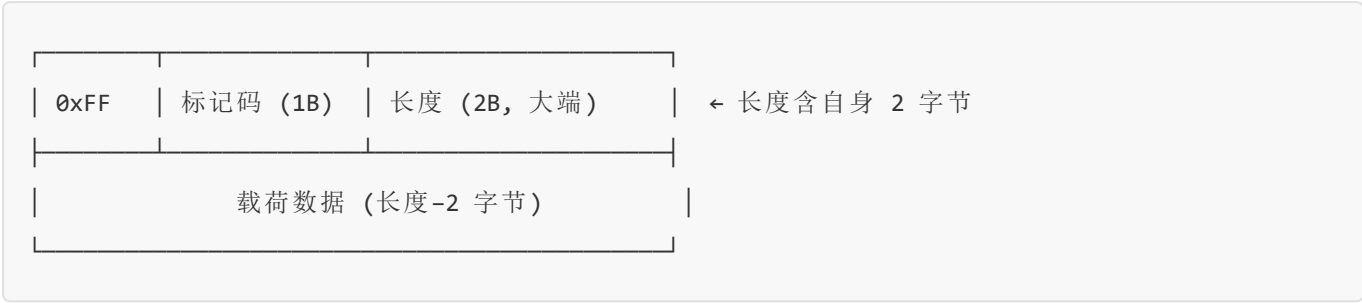
10. 算术编码（可选模式）

JPEG 扩展模式支持 **Q-coder** 算术编码（SOF9 / SOF10 / SOF11）。

对比项	霍夫曼	算术编码
压缩率	基准	提升约 5–10%
速度	快	慢约 2–3 倍
专利	无	历史上受专利限制（已过期）
普及度	极高	低（JPEG 2000 才真正普及算术编码）

11. 文件结构（标记段）

1. 标记段通用格式



2. 主要标记

标记	值	名称	说明
SOI	0xFFD8	Start of Image	文件开头
APP0	0xFFE0	JFIF 标识	版本、密度单位、缩略图
APP1	0xFFE1	EXIF / XMP	拍摄参数、GPS、缩略图
APP2	0xFFE2	ICC Profile	色彩配置文件
APP14	0xFFEE	Adobe	色彩变换标识 (YCbCr vs YCCK)
DQT	0xFFDB	Define Quantization Table	量化表 (可含 1–4 张)
DHT	0xFFC4	Define Huffman Table	霍夫曼表 (可含 1–4 张)
DAC	0xFFCC	Define Arithmetic Coding	算术编码条件
SOF0	0xFFC0	Baseline DCT	帧头：尺寸、分量数、采样因子
SOF2	0xFFC2	Progressive DCT	渐进帧头
DRI	0xFFDD	Define Restart Interval	重启间隔 (MCU 数)
SOS	0xFFDA	Start of Scan	扫描头：分量选择、表选择、谱选择
RSTn	0xFFD0–D7	Restart Marker	重启标记 (循环使用)
DNL	0xFFDC	Define Number of Lines	行数 (高度未知时)
COM	0xFFFE	Comment	注释
EOI	0xFFD9	End of Image	文件结尾

3. 典型文件布局

SOI	
APP0 (JFIF)	
APP1 (EXIF)	← 可选
APP2 (ICC)	← 可选
DQT (亮度量化表)	
DQT (色度量化表)	
SOF0 (帧头)	
DHT (DC 亮度表)	
DHT (AC 亮度表)	
DHT (DC 色度表)	
DHT (AC 色度表)	
SOS (扫描头)	
[熵编码数据 ...]	
EOI	

4. SOF0 帧头内容

字段	长度	说明
精度 P	1B	8 (基线) 或 12
高度 Y	2B	图像行数
宽度 X	2B	图像列数
分量数 Nf	1B	1 (灰度) / 3 (YCbCr) / 4 (CMYK)
分量 1	3B	ID=1, H=2, V=2, Tq=0 (Y, 4:2:0)
分量 2	3B	ID=2, H=1, V=1, Tq=1 (Cb)
分量 3	3B	ID=3, H=1, V=1, Tq=1 (Cr)

5. SOS 扫描头内容

字段	说明
分量数 Ns	本次扫描包含几个分量
分量选择	每个分量的 ID + DC 表号 + AC 表号
Ss	谱选择起始（顺序模式 = 0）
Se	谱选择结束（顺序模式 = 63）
Ah / Al	逐次逼近高位/低位（顺序模式 = 0/0）

12. 渐进式 JPEG (Progressive)

1. 原理

把图像分多次扫描编码，每次只传一部分信息，接收端逐次重建出越来越清晰的图像。

2. 两种渐进方式

方式	机制	效果
频谱选择 (Spectral Selection)	每次扫描传一部分 ZigZag 频段 ($S_s - S_e$)	先传低频轮廓，再补高频细节
逐次逼近 (Successive Approximation)	每次扫描传系数的一部分位 (A_h / A_l)	先传高位，再逐次补低位

3. 典型扫描脚本

扫描 1: Y 的 DC 系数 ($S_s=0, S_e=0, A_h=0, A_l=0$)
扫描 2: Cb/Cr 的 DC 系数
扫描 3: Y 的 AC 1-5 (低频)
扫描 4: Y 的 AC 6-63 (高频)
扫描 5: Cb/Cr 的 AC 1-63

4. 优缺点

优点	缺点
网络传输体验好 (先出模糊图)	文件略大 (约 5-10%)
支持"渐进渲染"	编解码更复杂
同等质量下感知更好	老解码器兼容性略差

13. 无损 JPEG (SOF3)

1. 原理

不做 DCT，直接用**预测编码**：用左、上、左上邻居预测当前像素，编码预测误差。

预测邻域布局

C | B

—+—

A | X ← X 为当前像素

2. 七种预测器

编号	预测公式
1	$X = A$
2	$X = B$
3	$X = C$
4	$X = A + B - C$
5	$X = A + \lfloor (B - C)/2 \rfloor$
6	$X = B + \lfloor (A - C)/2 \rfloor$
7	$X = \lfloor (A + B)/2 \rfloor$

3. 与 JPEG-LS 的区别

项	无损 JPEG (SOF3)	JPEG-LS (14495)
预测	固定 7 选 1	自适应 MED 预测
上下文建模	无	有 (365 个上下文)
压缩率	约 2:1	约 2:1–3:1 (更好)
普及度	低	医学影像常用

14. 重启间隔 (Restart Interval)

DRI 段指定每隔多少个 MCU 插入一个 RSTn 标记：

- 作用 1： **错误隔离**——码流损坏时最多影响一个间隔；
- 作用 2： **并行解码**——多线程可从 RSTn 处切分；
- 作用 3： **DC 预测器重置**——每个间隔开始 DC 预测器归零。

RST0 – RST7 循环使用（0xFFD0 → 0xFFD7 → 0xFFD0 ...）。

15. 典型伪影 (Artifacts)

伪影	成因	表现
块效应 (Blocking)	8×8 块独立量化，边界不连续	平坦区域出现网格状方块
振铃 (Ringing)	高频系数被粗量化，Gibbs 现象	强边缘附近出现波纹/鬼影
蚊噪 (Mosquito Noise)	同上，边缘周围的高频噪声	文字/线条边缘出现"蚊子飞舞"
色度渗漏 (Color Bleeding)	色度子采样	彩色边缘颜色溢出到邻近区域
渐变断层 (Banding)	平滑渐变被量化成台阶	天空等渐变区出现色带

缓解手段：提高质量因子、关闭色度子采样、使用渐进式、后处理去块滤波（deblocking）。

16. 解码流程

码流

- |
- └ 解析标记段 (SOI / APPn / DQT / SOF / DHT / SOS)
- |
- └ 熵解码 (霍夫曼 / 算术) → (RUN, SIZE) + 附加位
- |
- └ 反 DPCM 恢复 DC, 反 RLE 恢复 AC
- |
- └ 反 ZigZag → 8×8 系数矩阵
- |
- └ 反量化 × Q(u,v)
- |
- └ IDCT → 8×8 空间域块
- |
- └ +128 (反 Level Shift)
- |
- └ 色度上采样 (插值回全分辨率)
- |
- └ YCbCr → RGB → 显示

第二部分 JPEG 2000

1. 设计目标

JPEG 2000 (ISO/IEC 15444) 的目标是用一个统一框架覆盖有损 + 无损 + 渐进 + ROI:

目标	实现手段
无块效应	小波变换 (全局变换, 非分块)
有损/无损统一	5/3 可逆小波 + 9/7 不可逆小波
渐进传输	质量层 / 分辨率层 / 空间位置 三重渐进
感兴趣区域优先	ROI Maxshift / Scaling
抗误码	码块独立、可重同步
随机访问	码块 + 包结构

2. 编码总流程



1. 第①步：前处理 —— DC Level Shift 与分量变换

输入：tile 内的原始分量样本（8 位图像为 0~255 的无符号整数）。

(a) DC Level Shift（直流电平搬移）

$$s'(x,y) = s(x,y) - 2^{S_{siz}-1}$$

8 位图像的 $S_{siz} = 8$ ，故减去 $2^7 = 128$ ，样本被搬移到 $-128 \sim +127$ 。

- 为什么要做：像素本身是"单边"的非负量（均值约 128），DWT 之后 LL 子带会背上一个巨大的 DC 偏置，位平面编码要浪费大量比特去编码那个"全是 1 的高位"。搬移到 0 附近后 LL 系数幅度最小，

压缩率最高。

- **何时跳过**：如果 `SIZ` 标记里该分量的 `Ssiz` 最高位为 1（表示该分量本身就是**有符号**数据，例如某些医学影像、以及部分中间结果），标准规定**不做**电平搬移。
- **可逆性**：纯粹加减常数，无损。
- **解码端**：DWT 逆变换、反量化之后，加回 $2^{S_{siz}-1}$ 。

(b) 分量变换（仅多分量图像）

路径	变换	运算	能否精确可逆
无损 (5/3)	RCT 可逆色彩变换	整数	是
有损 (9/7)	ICT 不可逆色彩变换	浮点	否（浮点舍入）

公式见「分量变换」一节。灰度图（单分量）跳过此步。

关键：这一步不引入"结构性"损失（RCT 完全无损；ICT 只有极小的浮点舍入）。

2. 第②步：Tile 划分

- 参数：`SIZ` 标记里的 `XTsiz` / `YTsiz`（tile 宽高）与 `XT0siz` / `YT0siz`（参考网格偏移）。
- tile 是**独立编码单元**：DWT、量化、Tier-1、Tier-2 全部在 tile 内部闭合，tile 与 tile 之间不共享任何信息（边界靠对称延拓"假装"外侧有数据）。
- 为什么要切：限制编码/解码的内存峰值、支持并行、支持空间随机访问（只解需要的 tile）。
- 代价：tile 边界处滤波看不到"另一半"内容，可能出现极轻微的不连续；但因为完全没有 8×8 分块，仍远优于 JPEG 的块效应。
- 默认：`XTsiz=Xsiz`、`YTsiz=Ysiz`，即**整图一个 tile**。
- tile 内部还可再拆成 **tile-part**（`SOT` 标记），每个 tile-part 可以携带自己的 `COD` / `COC` / `QCD` / `QCC` / `POC`，这是"渐进传输 + 随机访问"的载体。

3. 第③步：小波变换（DWT）

- 对 tile 内**每个分量**分别做 N_L 级二维 DWT（先行后列或先列后行，结果等价）。
- 滤波器由 `COD` 的变换位决定：9/7（不可逆，默认）或 5/3（可逆）。
- 级数经验值：常见 $N_L = 5$ （最粗分辨率约 32 像素）；医学/遥感图像常用 5~6 级。
- 边界一律采用**对称延拓**（见「小波变换」一节的边界处理）：这是保证 5/3 精确可逆的必要条件。
- **输出**：1 个 $N_L LL$ 低频子带 + $3N_L$ 个高频子带（每层 HL / LH / HH 各一个）。

4. 第④步：量化

- 唯一的"主动"有损步骤。对每个子带的每个系数做**标量死区量化**（见「量化」一节）。
- 量化步长 Δ_b 由 QCD / QCC 中的 $R_b, \varepsilon_b, \mu_b$ 决定；也可以启用**推导量化**只传最深层 LL 的参数。
- 无损路径： $\Delta_b = 1$ ，量化索引 q 就等于整数小波系数本身，不丢任何信息。
- 视觉加权：标准允许给不同子带不同的 Δ_b （人眼对低频更敏感 → 低频步长更小）。
- **输出**：整数量化索引 $q_b(u, v)$ 。

5. 第⑤步：空间划分 —— subband → precinct → code-block

三层嵌套，作用是为"渐进传输 / 随机访问"建立索引：

subband（子带，DWT 产物）
└─ precinct（空间分区，默认每个分辨率层一个）
 └─ code-block（编码单位，默认 64×64）

- **precinct**：由 COD 的 PPx / PPy 指定（宽/高的 2 的幂指数），默认 15/15，即"整个分辨率层就是一个 precinct"。
- **code-block**：由 COD 的 xcb / ycb 指定指数，默认 64 × 64（可选 32 × 32、16 × 16）。
- 约束：code-block **不得跨 precinct**；同一分辨率层内三个子带的 code-block 划分必须一致（便于 packet 把同一空间位置的三个子带一起打包）。
- **输出**：每个子带被切成若干 code-block —— code-block 之间完全独立，是 Tier-1 的天然并行单位。

6. 第⑥步：EBCOT Tier-1（位平面编码 + MQ 算术编码）

对**每个 code-block** 独立执行：

1. 把系数写成 **符号-幅度** (sign-magnitude) 形式，得到该码块的位平面总层数 M_b ；
2. 从最高位平面（MSB）逐层向下扫到 LSB，每个位平面分三个**编码通道**：
3. **SPP** (Significance Propagation Pass)：把"已经显著"的信息向邻居传播；

4. **MRP** (Magnitude Refinement Pass) : 细化已经显著的系数;
5. **CUP** (Cleanup Pass) : 首次发现显著系数 (含 run-length 模式) ;
6. 每个比特用**上下文** (context) 查概率表后喂给 **MQ 算术编码器**: 零编码 ZC 9 个 + 符号编码 SC 5 个 + 幅度细化 MR 3 个 + 游程 RL 2 个 = **19 个上下文**;
7. 每个通道结束时记录一个**截断点**, 并给出该截断点的码率 R 与失真 D 。

输出: 每个 code-block 一段**可截断**的 MQ 码流 + 每个截断点的 (R_i^n, D_i^n) 数据 (交给 Tier-2) 。详见「EBCOT Tier-1」一节。

7. 第⑦步: EBCOT Tier-2 (率失真优化打包)

- 用 **PCRD-opt** (Post-Compression Rate-Distortion Optimization) 在**所有 code-block 的所有截断点**上做全局选择:

$$\min_{\{n_i\}} \sum_i (D_i^{n_i} + \lambda R_i^{n_i})$$

- 让 λ 由大到小扫一遍, 就得到一串**质量层** (quality layer) : 每多一个层, 就多花一点码率、多降一点失真。
- 每个 packet 里"哪些 code-block 出现 / 各自截断在第几个字节 / 数据从哪里开始", 用 **tag tree** 压缩后写进 **packet header**。

详见「EBCOT Tier-2」一节。

8. 第⑧步: 码流组织

```
packet  → layer (按质量分层)
        → tile-part (SOT ... SOD)
        → codestream (SOC ... EOC)
```

- 排列顺序由**渐进顺序**决定: LRCP / RLCP / RPCL / PCRL / CPRL (见「三重渐进性」一节) 。同一堆 packet, 换个顺序就能得到"先清晰后细化 / 先中心后边缘 / 先低分辨率后高分辨率"等不同的渐进体验。
- 最后套上 **SOC**、**SIZ**、**COD**、**QCD** ...、**SOT**、**SOD**、**EOC** 等标记段 (见「文件格式」一节) 。

9. 八步速览

步	处理	输入 → 输出	关键参数 / 标记	有损?
①	DC Level Shift + 分量变换	原始样本 → 零均值分量	SIZ.Ssiz、RCT/ICT	否 (ICT 仅浮点舍入)
②	Tile 划分	图像 → tiles	SIZ.XTsiz/YTsiz	否
③	DWT	tile → 子带	COD 变换位、级数 N_L	否 (9/7 仅浮点舍入)
④	量化	子带 → 整数 q	QCD / QCC	是 ($\Delta_b > 1$ 时)
⑤	划分	子带 → precinct → code-block	COD 的 PPx/PPy、xcb/ycb	否
⑥	Tier-1	code-block → 可截断码流	19 个上下文、MQ	否
⑦	Tier-2	截断点 → 质量层 / packet	PCRD-opt、tag tree	是 (截断即丢信息)
⑧	码流组织	packet → codestream	渐进顺序、标记段	否

3. 分量变换

1. RCT (可逆色彩变换, Reversible Color Transform)

用于**无损**路径, 整数运算:

$$Y = \left\lfloor \frac{R + 2G + B}{4} \right\rfloor$$

$$C_b = B - G$$

$$C_r = R - G$$

逆变换:

$$G = Y - \left\lfloor \frac{C_b + C_r}{4} \right\rfloor$$

$$R = C_r + G$$

$$B = C_b + G$$

2. ICT (不可逆色彩变换, Irreversible Color Transform)

用于**有损**路径, 浮点运算, 基于 Rec.601:

$$Y = 0.299R + 0.587G + 0.114B$$

$$C_b = -0.16875R - 0.33126G + 0.5B$$

$$C_r = 0.5R - 0.41869G - 0.08131B$$

注意: JPEG 2000 的 Cb/Cr **不加 128 偏移** (内部用有符号表示)。

3. RGB 到底要不要先平移？—— 先把两个"128"分清

结论：RGB 不平移。

Rec.601 变换里的 +128 只加在

Cb/Cr

上，而且它是"偏置 (bias) "，不是"平移 RGB"。

Y 保持 0~255，并没有变成 ±127。

真正把样本搬移到零均值的，是编码流程第 ④ 步的 Level Shift (−128)，它作用在色彩变换之后的 Y/Cb/Cr 三个分量上。

对比项	色彩变换里的 +128	Level Shift 的 −128
所在位置	第 ① 步 (色彩变换)	第 ④ 步 (FDCT 之前)
作用对象	只有 Cb、Cr	Y、Cb、Cr 全部
方向	+128	−128
目的	把双极性色差搬进无符号 0~255	把单边像素搬成零均值，降低 DC 幅度
是否可逆	是 (解码时执行 −128)	是 (解码时执行 +128)
性质	变换公式的组成部分	独立的处理步骤
对 Y 的影响	无	Y 被搬移到 −128~127

上表描述的是 JPEG 的做法。JPEG 2000 的 Cb/Cr 不加偏置，顺序也正好相反，见本节第 8 小节。

完整数据流：

RGB 图像 (0~255)



① 色彩变换 (Rec.601) — RGB 原样代入, 不平移

$Y = 0.299R + 0.587G + 0.114B \rightarrow 0\sim255$

$Cb = 128 - 0.168736R - 0.331264G + 0.5B \rightarrow 0\sim255$ (128 = 中性灰)

$Cr = 128 + 0.5R - 0.418688G - 0.081312B \rightarrow 0\sim255$ (128 = 中性灰)

Y (0~255) Cb (0~255) Cr (0~255)

④ Level Shift: 每个分量各自 -128

Y (-128~127) Cb (-128~127) Cr (-128~127)

8x8 分块 → FDCT → 量化 → ZigZag → 熵编码 ...

注意：Y 在色彩变换后仍是 0~255，是第 ④ 步才把它变成 -128~127 的。所以"输入 0~255 的 RGB 自动变成 ±127 的图像"这个说法不准确 —— 只有走完第 ④ 步才会变成 ±127，而且是三个分量一起变的。

4. 数值验证：从 RGB 到 ±127 的完整演变

颜色	R, G, B	Y	Cb	Cr	Y-128	Cb-128	Cr-128
黑	0, 0, 0	0	128	128	-128	0	0
白	255, 255, 255	255	128	128	127	0	0
中灰	128, 128, 128	128	128	128	0	0	0
纯红	255, 0, 0	76	85	255	-52	-43	127
纯绿	0, 255, 0	150	44	21	22	-84	-107
纯蓝	0, 0, 255	29	255	107	-99	127	-21

观察：

- 第 3~5 列 (Y / Cb / Cr) 全部落在 0~255，中性灰恰好落在 128 —— 说明色彩变换后并没有出现负值；
- 最后三列 (各减 128) 才全部落进 -128~127 —— 这才是 Level Shift 的功劳；

- 纯色的 Y 只有 76 / 150 / 29, 远不到 255 —— 这正是"转 YCbCr 会损失一部分动态范围"的原因, 也是为什么 Y 用精细量化表而 Cb/Cr 可以用粗表。

5. 数学解释: 为什么"不平移 RGB"是合法的

先看三条公式的系数和:

$$0.299 + 0.587 + 0.114 = 1 \quad (Y)$$

$$-0.168736 - 0.331264 + 0.5 = 0 \quad (C_b)$$

$$0.5 - 0.418688 - 0.081312 = 0 \quad (C_r)$$

这带来一个漂亮的性质: **色差信号对 RGB 的整体直流电平完全免疫**。假设把 R、G、B 同时减去 128:

$$\begin{aligned} C'_b &= 128 + \sum_i c_i (X_i - 128) = C_b + 128 \underbrace{\sum_i c_i}_{=0} = C_b \\ Y' &= \sum_i y_i (X_i - 128) = Y - 128 \underbrace{\sum_i y_i}_{=1} = Y - 128 \end{aligned}$$

即:

"先平移 RGB 再变换" $\equiv$ "变换后只平移 Y", Cb/Cr 完全不变。

两种做法数学上等价, `+128` 偏置与 `-128` 搬移只是两个独立可交换的操作。差别只在于 Y 在何时变成有符号量。

6. 为什么标准选择"不平移 RGB"而不是"平移 RGB"

理由	说明
保持 Y 的物理意义	Y=0 是黑、Y=255 是白，便于显示、调试、与视频标准对接
少一次运算	反正第 ④ 步要统一减 128，没必要在色彩变换前先减一遍
Cb/Cr 的偏置是必须的	色差本身是双极性的（-128~+127），而 JPEG 的样本必须是无符号 8 位，所以必须加偏置
与硬件/视频链路对齐	前面接的是 BT.601 视频管线，那里 Y 就在 0~255（或 16~235）

7. 澄清："128-127"是一个误读

"128-127"应理解为"以 128 为中心、跨度 ±127.5"，而不是"取值范围是 128 到 127"。

- Cb/Cr 加 128 偏置之后是 0~255，中性点 128；
- 去掉偏置后色差本身是 -128~+127（连续域上是 -127.5~+127.5）；
- Level Shift 之后三个分量都是 -128~+127。

另外，色差的实际取值范围比理论值窄得多：Cb 的理论极值是 0.5~255.5（即 -127.5~+127.5），但真实图像几乎不会触及，这也是标准色度量化表可以放得很粗的原因之一。

8. JPEG 与 JPEG 2000 的处理顺序正好相反

	JPEG	JPEG 2000
处理顺序	色彩变换 → Level Shift	Level Shift → 分量变换
色彩变换输出	无符号 0~255（Cb/Cr 带 +128 偏置）	有符号（Cb/Cr 不加偏置）
色差定义	$Cb = 128 - 0.168736R - \dots$	RCT: $Cb = B - G$ （纯差值）
顺序成因	与 +128 偏置耦合	RCT/ICT 天然输出有符号差值，必须先搬移再变换
可逆性	浮点，不可逆	RCT 精确可逆 / ICT 不可逆

JPEG: RGB(0~255) —色彩变换→ Y,Cb,Cr(0~255) —128→ ±127
 JPEG 2000: R,G,B(0~255) —128→ R,G,B(-128~127) —分量变换→ Y,Cb,Cr(有符号)

为什么 JPEG 2000 必须反过来？ 看 RCT 的色差公式 $C_b = B - G$ 、 $C_r = R - G$ （以及 ICT 的 $C_b = -0.16875R - 0.33126G + 0.5B$ ）：它们的系数和也是 0，**输出天然围绕 0 分布的有符号差值**，不存在"需要 +128 才能放进无符号容器"的约束（JPEG 2000 内部允许有符号整数表示）。

反过来说，JPEG 之所以要 +128，是因为 **8-bit JPEG 码流的样本字段定义为无符号**，色差必须先加偏置才能写入。于是 Level Shift 只能放在**最后**（FDCT 之前），对已经无符号的 Y/Cb/Cr 统一减 128。

两个标准的 Level Shift 目的相同（把单边数据搬成零均值、降低 LL/DC 系数幅度），只是**插入位置被色彩变换的输出符号性决定了**。

9. full range 与 studio swing

同样是 8-bit，Y/Cb/Cr 的"有效范围"有两种约定：

约定	Y	Cb / Cr	用在哪
Full range (JPEG / JFIF)	0~255	0~255	JPEG 文件、消费级图像
Studio swing (BT.601 视频)	16~235	16~240	SDI / MPEG / H.264 等广播链路

Studio swing 留出"脚下空间（footroom）"和"头上空间（headroom）"，供同步信号与超白/超黑使用。两者的换算：

studio → full:

$$Y_{full} = (Y_{studio} - 16) \times \frac{255}{219}$$

$$C_{full} = (C_{studio} - 128) \times \frac{255}{224} + 128$$

full → studio:

$$Y_{studio} = Y_{full} \times \frac{219}{255} + 16$$

$$C_{studio} = (C_{full} - 128) \times \frac{224}{255} + 128$$

把 studio swing 的数据当成 full range 解释，画面会**变亮、变灰、对比度下降**（因为 16→0、235→255 被拉伸）；反过来则**发暗、发闷**。这就是"某些 JPEG 解码后颜色偏灰或偏艳"的常见原因。

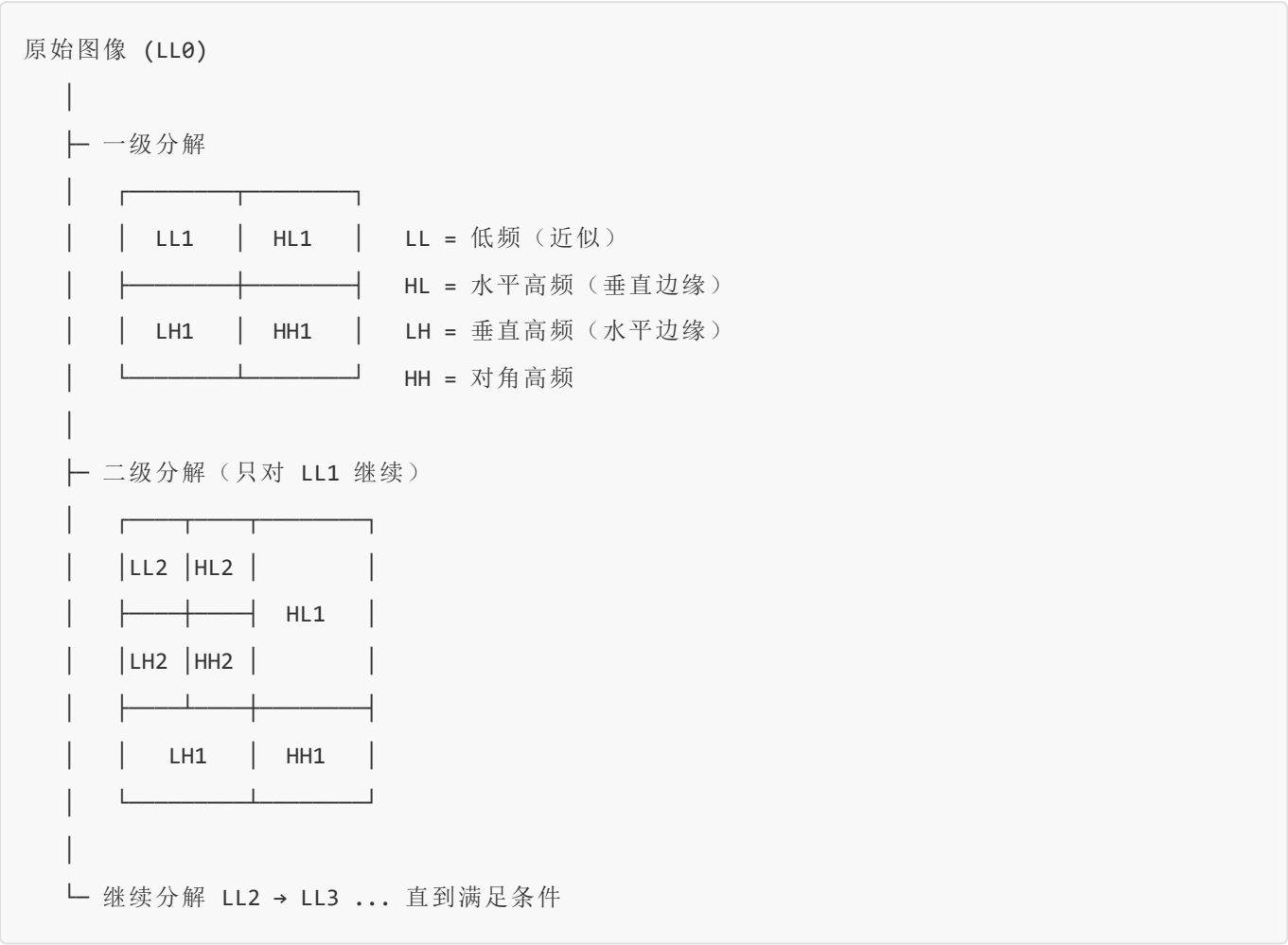
注意：JPEG 文件规范（JFIF）默认 full range，但标准**不强制**——有些编码器会把 studio swing 数据直接写进 JPEG，解码器无从判断，只能靠约定。

4. 分块 (Tile)

- 图像可切成矩形 tile，每个 tile **独立编码**（类似 JPEG 的块，但大得多）；
 - 默认整图一个 tile；
 - 作用：限制内存占用、支持并行、支持空间随机访问；
 - 代价：tile 边界可能出现轻微不连续（但远小于 JPEG 的 8×8 块效应）。
-

5. 小波变换（DWT）

1. 多级分解结构



子带命名：第一个字母 = 水平方向（L 低通 / H 高通），第二个字母 = 垂直方向。

2. 两种小波滤波器

名称	类型	滤波器长度	用途
CDF 9/7	不可逆（浮点）	9 抽头分析 / 7 抽头合成	有损压缩（默认）
Le Gall 5/3	可逆（整数）	5 抽头分析 / 3 抽头合成	无损压缩

3. CDF 9/7 提升系数

参数	值
α	-1.586134342
β	-0.05298011854
γ	0.8829110762
δ	0.4435068522
K	1.230174105 (缩放)
$1/K$	0.812893066

4. Le Gall 5/3 提升公式

预测步（奇数样本）：

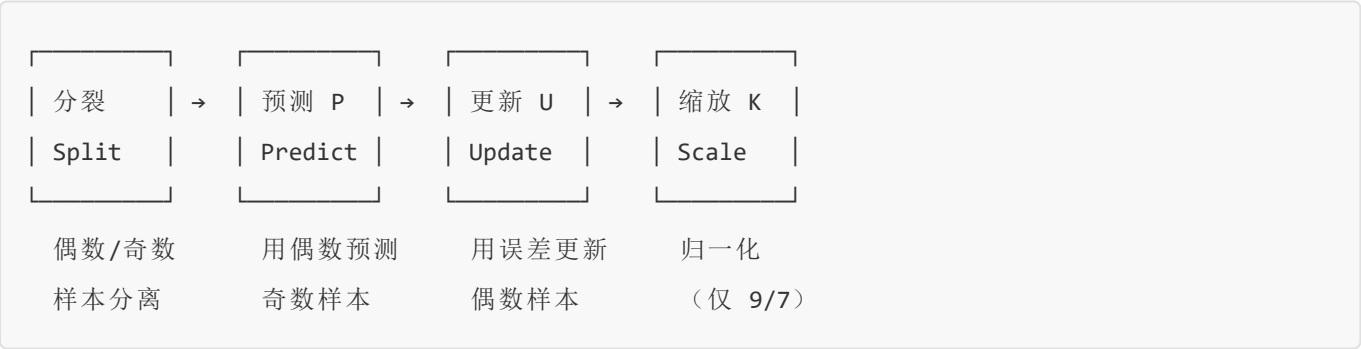
$$d[n] = d[n] - \left\lfloor \frac{s[n] + s[n + 1]}{2} \right\rfloor$$

更新步（偶数样本）：

$$s[n] = s[n] + \left\lfloor \frac{d[n - 1] + d[n] + 2}{4} \right\rfloor$$

5. 提升方案（Lifting Scheme）

把滤波器分解为一系列"预测 + 更新"步骤：



1. 提升与 DWT / FWT 是什么关系

这三个词经常被混在一起讲，其实是三个不同层次的东西：

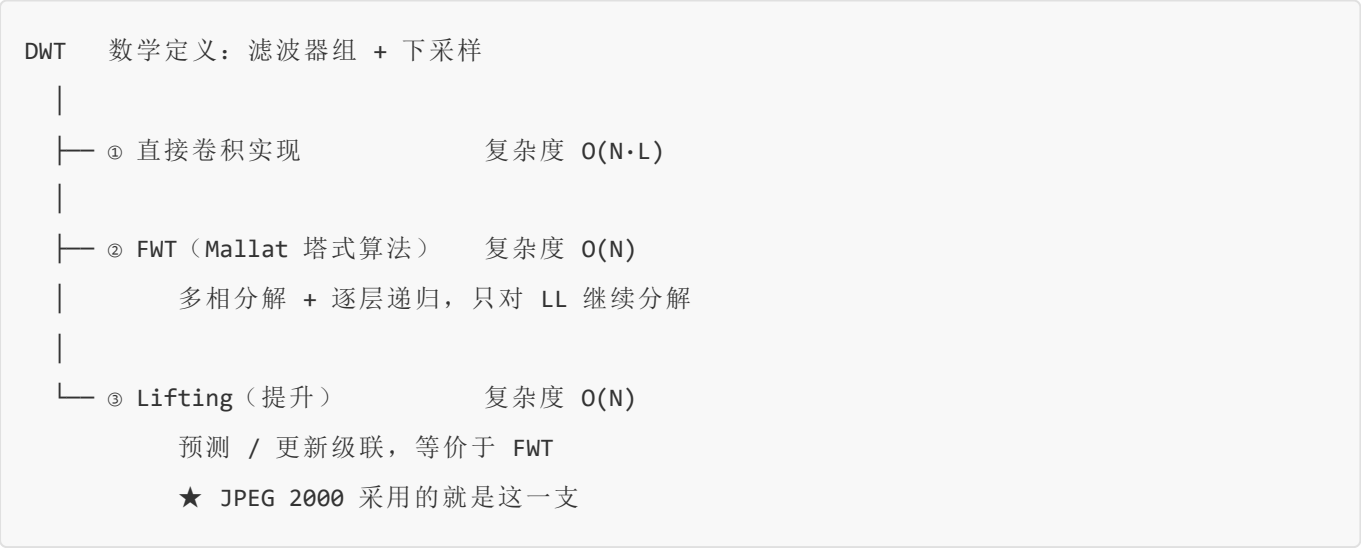
名称	层次	定义
DWT (Discrete Wavelet Transform, 离散小波变换)	数学定义	用一对滤波器（低通 h 、高通 g ）对信号做卷积 + 下采样，得到低频近似 a 与高频细节 d
FWT (Fast Wavelet Transform, 快速小波变换)	算法	用多相分解 + 递归的方式把 DWT 快速算出来，复杂度 $O(N)$ ；与「FFT 之于 DFT」的关系完全对应
Lifting (提升方案)	另一种算法结构	把滤波器组分解成一系列“预测 / 更新”初等步骤，等价于 FWT，但更快、可原位、可整数可逆

DWT 的分辨率分析定义式（分析滤波器组）：

$$a[n] = \sum_k h[2n - k] x[k], \quad d[n] = \sum_k g[2n - k] x[k]$$

直接按定义算，每个输出点要做 L 次乘加（ L = 滤波器长度），整层复杂度 $O(NL)$ ；FWT 把它降到 $O(N)$ ；提升则进一步把常数因子压小。

三者关系图：



核心结论：提升不是“另一种小波”，而是“同一个双正交小波的另一算法”。Daubechies 与 Sweldens (1996) 证明了：

任何具有有限冲激响应的双正交小波滤波器组，都可以分解为有限个提升步骤的级联。

由此得到两个重要推论：

- 1. 用卷积算 9/7 与用提升算 9/7，**数学结果完全等价**（只差浮点舍入）；
- 2. 用卷积算 5/3 **无法**保证整数可逆，只有用提升（整数提升 + 取整）才能做到**逐位无损**。

所以 JPEG 2000 明确规定：变换必须以提升形式实现。

6. 提升为什么优于直接卷积

维度	直接卷积	提升
计算量	每输出点 L 次乘加	乘法次数约减半
内存	需缓存整行 / 整列	原位计算 (in-place)
整数可逆	✗ 浮点卷积无法取整还原	✓ 5/3 用 $\lfloor \cdot \rfloor$ 实现
边界处理	需显式做对称延拓，容易写错	延拓自然融入预测 / 更新
并行性	好	好（但依赖链更长）
硬件实现	需要完整乘法器	5/3 只需加法 + 移位

7. 提升的三步骨架与公式

- **Split (分裂)**：把信号按奇偶下标拆成 x_e （偶数样本）与 x_o （奇数样本）。这一步本身不做任何滤波，称为**惰性小波**（lazy wavelet）。
- **Predict (预测)**：用偶数样本预测奇数样本，预测误差就是高频细节

$$d[n] = x_o[n] - P(x_e)[n]$$

- **Update (更新)**：用刚算出的细节反过来修正偶数样本，使其成为真正的低频近似

$$a[n] = x_e[n] + U(d)[n]$$

- **Scale (缩放)**：**可选**，只有 9/7 需要，把低频 / 高频能量拉平（见「缩放」一节）。

对于 Le Gall 5/3，两个算子就是简单的平均：

$$P(x_e)[n] = \left\lfloor \frac{x_e[n] + x_e[n + 1]}{2} \right\rfloor, \quad U(d)[n] = \left\lfloor \frac{d[n - 1] + d[n] + 2}{4} \right\rfloor$$

"接力赛"直觉：

- **预测**：偶数点当"队长"，去猜奇数点（细节）。奇数点真实值减去预测值，得到残差——这就是高频细节；
- **更新**：刚算出的残差反过来修正偶数点，让偶数点从"原始采样点"变成真正的**局部平均值**（低频平滑信号）。

8. 逆变换：提升为什么天然可逆

提升的每一步都是 $y = x + f(\text{其他})$ 的形式，逆运算就是 $x = y - f(\text{其他})$ ——**不要求逆矩阵、不需要除法**，只要"其他量"在逆变换时已经被恢复出来即可。

正变换	逆变换
Split: $x \rightarrow (x_e, x_o)$	Merge: $(a, d) \rightarrow x$
Predict: $d = x_o - P(x_e)$	$x_o = d + P(x_e)$
Update: $a = x_e + U(d)$	$x_e = a - U(d)$
Scale: $d = (-K) \cdot d, \quad a = a/K$	$d = d/(-K), \quad a = a \cdot K$

关键：正变换若全程使用**整数运算 + 确定性取整**，逆变换就能逐位还原——这就是 5/3 无损的数学基础。反过来，9/7 的系数是无理数，浮点乘加必然带舍入，所以不可逆。

9. 缩放（Scaling）：9/7 独有的"能量均衡器"

为什么 5/3 不需要缩放，9/7 却必须缩放？

9/7 经过前 4 步提升后，低频子带系数幅度变得**非常小**、高频子带幅度变得**非常大**。如果不缩放：

- 高频系数占用极大的动态范围，量化步长不好选；
- 不利于位平面编码（高位平面容易出现"整片 0 或整片 1"）。

缩放的作用：

子带	缩放	数值
低频 (a)	$\times 1/K$	≈ 0.812
高频 (d)	$\times (-K)$	≈ -1.230

目的：把低频与高频的**能量拉平到同一数量级**，这样量化器 ($R_b, \varepsilon_b, \mu_b$ ，见「量化」一节) 就能用统一标准处理，编码效率最高。

因为缩放乘的是**无理数**，它进一步坐实了 9/7 的"有损"属性——无损路径绝对不能出现这种改变数值量级的浮点乘法。

10. 5/3 与 9/7 提升步骤逐步对照

步骤	Le Gall 5/3 (可逆)	CDF 9/7 (不可逆)
Split	奇偶分裂	奇偶分裂
预测 1	$d[n] -= \lfloor (s[n] + s[n + 1])/2 \rfloor$	$d[n] += \alpha (s[n] + s[n + 1])$
更新 1	$s[n] += \lfloor (d[n - 1] + d[n] + 2)/4 \rfloor$	$s[n] += \beta (d[n - 1] + d[n])$
预测 2	—	$d[n] += \gamma (s[n] + s[n + 1])$
更新 2	—	$s[n] += \delta (d[n - 1] + d[n])$
缩放	无	$d *= -K, s /= K$
取整	有 ($\lfloor \cdot \rfloor$)	无
可逆	✅ 100% 还原	❌ 舍入不可逆

5/3 全程只有 2 步、0 个乘法；9/7 有 6 步、其中 2 步是缩放。

11. 5/3 为什么能"完美可逆"

两个关键点：

- 1. **没有缩放步骤**——不改变数值量级，不引入无理数；
- 2. **有向下取整 $\lfloor \cdot \rfloor$** ——这是无损的灵魂。取整虽然丢掉了小数部分，但这个丢失是**确定性的、可重复的**。

逆变换（严格逆着推回去）：

$$s[n] = s[n] - \left\lfloor \frac{d[n - 1] + d[n] + 2}{4} \right\rfloor$$

（先解更新）

$$d[n] = d[n] + \left\lfloor \frac{s[n] + s[n + 1]}{2} \right\rfloor$$

（再解预测）

因为整数加减和取整完全可逆，原始像素被 **100% 完美还原**，没有任何误差。

代价：5/3 的**消失矩**低，频带分离不够干净，能量集中度不如 9/7，所以压缩率相对较低。

12. 9/7 为什么不可逆，却仍然被采用

不可逆的原因：

- $\alpha, \beta, \gamma, \delta, K$ 全是**无理数**（浮点）；
- 没有取整，但计算机浮点精度有限（双精度只有 15–17 位有效数字），无理数乘加必然产生舍入误差；
- **更致命的一点**：图像像素是整数，经过这几步浮点运算后变成浮点系数；解码端即使完美逆推，最后算出的像素仍是浮点数，**必须四舍五入回整数**——这个舍入不可逆。

仍然采用的理由：它拥有 **4 阶消失矩**。对于图像中平滑的区域（天空、皮肤、大色块），它能极其精准地预测，残差几乎为 0；能量高度集中在 LL 子带，高频子带几乎全是 0。量化时可以大刀阔斧地砍掉高频，压缩率极高，且视觉上几乎看不出失真。

13. 消失矩 (Vanishing Moments)

若小波具有 p 阶消失矩，则它对任意 $p - 1$ 次多项式信号的高频响应为 0。

小波	消失矩阶数	含义
Haar	1	只能精确表示常数信号
Le Gall 5/3	2	能精确表示线性信号
CDF 9/7	4	能精确表示三次多项式信号

消失矩越高 → 平滑区域残差越小 → 能量越集中 → 压缩率越高。代价是滤波器更长（计算量更大、边界效应更明显）。

14. 边界处理

小波滤波需要超出边界的样本，JPEG 2000 规定用**对称延拓**（symmetric extension）：

... c b a | a b c d e f g h | h g f ...

↑ 左边界对称延拓 ↑ 右边界对称延拓

6. 量化

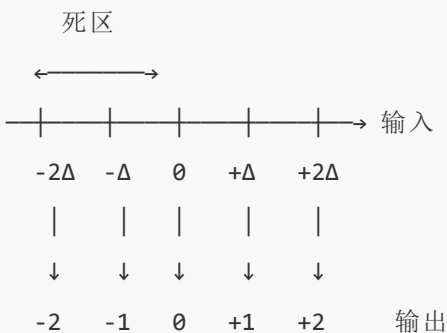
1. 标量量化

$$q_b(u, v) = \text{sign}(a_b(u, v)) \cdot \left\lfloor \frac{|a_b(u, v)|}{\Delta_b} \right\rfloor$$

其中 Δ_b 是子带 b 的量化步长。

2. 死区量化 (Deadzone)

量化区间示意 (Δ = 步长, 死区宽度 = 2Δ)



死区 (deadzone) 比普通量化区间宽一倍, 使小幅值系数更容易被量化为 0, 提高压缩率。

3. 量化步长与动态范围

核心公式:

$$\Delta_b = 2^{R_b - \epsilon_b} \times \left(1 + \frac{\mu_b}{2^{11}}\right)$$

三个参数分工明确:

参数	角色	位宽	是否传输
R_b	理论天花板（该子带最多需要多少比特）	—	❌ 双方默认
ε_b	粗调指数（决定用哪个 2 的幂作基数）	5 位（0~31）	✅
μ_b	细调尾数（在相邻两个 2 的幂之间平滑过渡）	11 位（0~2047）	✅

1. R_b ：理论动态范围（理论天花板）

$$R_b = R_s + \log_2 G_b$$

- R_s ：源图像精度。8 位图像 $R_s = 8$ 。
- G_b ：子带增益。LL = 1, HL / LH = 2, HH = 4。

8 位图像示例：

子带	G_b	R_b	理论最大范围
LL	1	8	约 0 ~ 255
HL / LH	2	9	约 -510 ~ 510
HH	4	10	约 -1024 ~ 1024

关键理解： R_b 不是"实际最大值减最小值"，而是"**理论上最多需要多少比特来表示**"。它是固定的、双方默认知道的，编码器**不需要传输**它。

2. ε_b ：粗调指数（调节阀）

$$\text{粗调基数} = 2^{R_b - \varepsilon_b}$$

ε_b 是 5 位指数，范围 0~31。

- ε_b 越大 → 粗调基数越小 → 量化越**细**；
- ε_b 越小 → 粗调基数越大 → 量化越**粗**。

3. μ_b ：细调尾数（微调器）

$$\text{乘数} = 1 + \frac{\mu_b}{2^{11}}$$

μ_b 是 11 位尾数，范围 0~2047。

- 乘数范围 **1.0 ~ 2.0**，精度 $1/2048 \approx 0.05\%$ ；

- 它让量化步长可以在两个相邻的 2 的幂之间平滑过渡。

4. 死区宽度

死区由量化公式自动产生： $|c| < \Delta_b \Rightarrow q = 0$ ，所以死区范围是 $(-\Delta_b, \Delta_b)$ ，宽度 $2\Delta_b$ 。

编码器通过动态选择 ε_b 和 μ_b ，保证死区不会过大而清空图像，也不会过小而浪费码率。

5. 完整实例：从理论到实践

设定背景：8 位灰度图像，9/7 小波一级分解，聚焦 HH（对角细节）子带。

- $R_s = 8$ ，HH 子带增益 $G_b = 4$ ；
- $R_b = 8 + \log_2 4 = 10$ ，理论范围 $-1024 \sim 1024$ 。

实际系数（编码器看到的真实数据）：

```
[ 5, -12, 200, 512 ]
[ -8, 15, -50, 300 ]
[ 10, -18, 400, -250 ]
[ 25, -5, 100, -400 ]
```

特点：大部分系数较小（5、8、10、12），少数边缘系数极大（512、400）。

编码器决策：目标量化步长 $\Delta_b = 10$ 。

1. 找粗调基数：希望基数在 10 附近。因 $R_b = 10$ ，取 $\varepsilon_b = 7 \rightarrow \text{基数} = 2^{10-7} = 2^3 = 8$ ；
2. 确定细调乘数： $10/8 = 1.25$ ；
3. 反推尾数： $1 + \mu_b/2048 = 1.25 \Rightarrow \mu_b = 512$ ；
4. 合法性检查： $\varepsilon_b = 7 \in [0, 31]$ ， $\mu_b = 512 \in [0, 2047]$ .

写入码流： $\varepsilon_b = 7$ （5 位二进制 00111）、 $\mu_b = 512$ （11 位二进制 01000000000）。实际 $\Delta_b = 10$ ，死区 $(-10, 10)$ 。

量化计算 ($q = \text{sign}(c) \times \lfloor |c|/10 \rfloor$)：

实际系数 c	$ c $	$ c /\Delta_b$	向下取整	符号	量化索引 q	判定
5	5	0.5	0	+	0	落入死区，被清空
-8	8	0.8	0	-	0	落入死区，被清空
15	15	1.5	1	+	1	越过死区，保留
-50	50	5.0	5	-	-5	保留
512	512	51.2	51	+	51	保留核心边缘

死区的作用：幅度为 5 和 -8 的微小波动，因为绝对值小于 10，被死区"一棍子打死"变成 0，极大有利于后续 EBCOT 压缩；而幅度 15 以上的系数被保留，图像结构并未丢失。

解码与反量化：

$$\hat{c} = \begin{cases} 0 & q = 0 \\ \text{sign}(q) \times (|q| + 0.5) \times \Delta_b & q \neq 0 \end{cases}$$

原始 c	q	重建公式	重建值 $\hat{c}$	误差
5	0	直接为 0	0	5
-8	0	直接为 0	0	8
15	1	1.5×10	15	0
-50	-5	-5.5×10	-55	5
512	51	51.5×10	515	3

小系数被当作 0 丢弃（有误差），但大系数（512）被完美保留，重建为 515，误差极小。

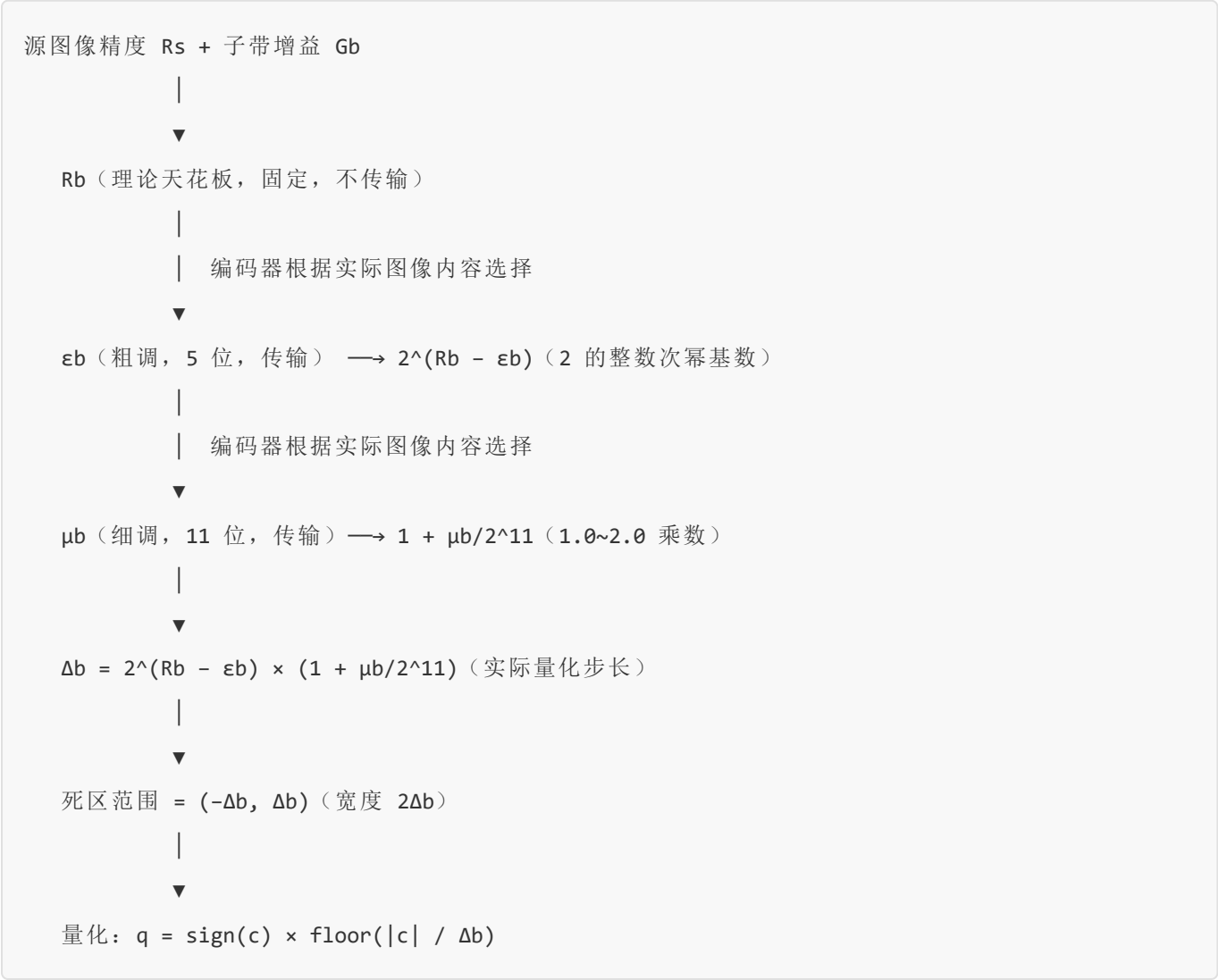
6. 反面教材： ε_b 选错会怎样

假设编码器"偷懒"选了 $\varepsilon_b = 0$ ，则 $\Delta_b = 2^{10-0} = 1024$ ，死区 $(-1024, 1024)$ 。

此时不管是 5、15 还是 512，**全部落在死区内**，全部变成 $q = 0$ 。解码后整个 HH 子带全是 0，图像的高频细节被彻底抹平，变成模糊的色块。

但编码器的**率失真优化（RDO）**绝不会允许这种极端情况发生，它会自动把 ε_b 调到合适的值（比如这里的 7）。

7. 参数关系图



8. 参数角色总结

参数	角色	比喻	是否传输
R_b	理论天花板	仓库最大容量，固定不变	不传输，双方默认
ε_b	粗调	决定用哪个大箱子装	传输（5 位）
μ_b	细调	决定箱子里的填充物怎么塞	传输（11 位）
Δ_b	实际步长	最终实际使用的箱子大小	计算得到
死区	自动产生	$ c < \Delta_b$ 的系数全部归零	由公式自动决定

一句话总结： R_b 画了一个大圈（理论范围）， ε_b 和 μ_b 在这个大圈里精确地找到实际需要的量化步长 Δ_b ，而死区 $(-\Delta_b, \Delta_b)$ 是量化公式自动产生的，宽度是 $2\Delta_b$ 。

4. 无损 vs 有损：量化参数怎么定

1. 无损压缩 (Error-free) 的参数确定

标准原文：

"For error-free compression, $\mu_b = 0$, $R_b = \varepsilon_b$, and $\Delta_b = 1$."

这是数学推导的必然结果。回顾公式：

$$\Delta_b = 2^{R_b - \varepsilon_b} \times \left(1 + \frac{\mu_b}{2^{11}}\right)$$

要让量化步长严格等于 1 (无损)，必须同时满足两个条件：

1. **指数部分必须为 1**: $2^{R_b - \varepsilon_b} = 1 \Rightarrow R_b - \varepsilon_b = 0 \Rightarrow \varepsilon_b = R_b$;
2. **尾数部分必须为 1**: $1 + \mu_b/2^{11} = 1 \Rightarrow \mu_b = 0$ 。

结论：无损模式下编码器**不需要计算、也不需要传输** ε_b 和 μ_b 。解码器默认知道 $\mu_b = 0$ ，且 ε_b 直接等于该子带的 R_b (双方提前约定的固定值)。此时 $\Delta_b = 1$ ，量化索引 q 就是整数小波系数本身，没有任何信息丢失。

2. 有损压缩 (Irreversible) 的参数确定

标准原文：

"For irreversible compression, no particular quantization step size is specified in the standard."

标准不规定具体量化步长，完全由编码器根据自己的**码率控制、视觉加权、率失真优化 (RDO)** 自由决定。问题变成：如何把这个步长告诉解码器？标准提供两种参数传递模式。

模式一：显式量化 (Expounded Quantization)

标准原文：

"Instead, the number of exponent and mantissa bits must be provided to the decoder on a subband basis, called expounded quantization"

- **做法**：编码器为图像里的**每一个子带** (LL、HL、LH、HH) 分别计算并选择最合适的 ε_b 和 μ_b ；
- **传输**：把这些参数明确写进码流的 **QCD** (默认量化) 或 **QCC** (分量量化) 标记段；
- **解码**：解码器直接读取每个子带的 ε_b 和 μ_b ，代入公式算出各自**完全独立**的 Δ_b ；
- **特点**：最灵活、压缩效果最好，但需要传输的参数最多（每个子带 16 位，子带多时仍有开销）。

模式二：推导量化 (Derived Quantization)

标准原文：

"or for the NLLL subband only, called derived quantization. In the latter case, the remaining subbands are quantized using extrapolated NLLL subband parameters."

- **做法**：编码器只计算并传输**最深层的低频子带** $N_L LL$ 的参数（记为 ε_0 和 μ_0 ）；
- **外推**：其他高频子带 (HL、LH、HH) 不单独传输参数，而是通过**外推公式 (标准公式 8-64)** 得到：

$$\mu_b = \mu_0, \quad \varepsilon_b = \varepsilon_0 + n_b - N_L$$

其中 N_L 是小波分解的总级数， n_b 是当前子带 b 所在的分解层数。

- **目的**：进一步节省传输开销，牺牲少许灵活性换取码流中量化参数传输量的大幅下降。

3. 深入理解推导量化公式 (8-64)

回顾 $R_b = R_s + \log_2 G_b$ 。高频子带的增益 G_b 随分解层数增加而变大。设总级数 $N_L = 3$ ：

子带	增益 G_b	R_b
3LL	1	R_s
2HL / 2LH	2	$R_s + 1$
1HL / 1LH	4	$R_s + 2$
1HH	8	$R_s + 3$

推导量化希望所有子带的量化步长 Δ_b 保持某种**视觉上的一致性**。由于 R_b 随层数变大，为了让 $2^{R_b - \varepsilon_b}$ 保持不变（即 Δ_b 大致相同）， ε_b 也必须跟着变大。

公式 $\varepsilon_b = \varepsilon_0 + n_b - N_L$ 正好实现了这个目的：

- 3LL: $n_b = 3 \rightarrow n_b - N_L = 0 \rightarrow \varepsilon_b = \varepsilon_0$ ；
- 1HH: $n_b = 1 \rightarrow n_b - N_L = -2 \rightarrow \varepsilon_b = \varepsilon_0 - 2$ 。

由于 R_b 在 1HH 处比 3LL 处大了 3（增益 8 vs 1），而 ε_b 减小了 2， $R_b - \varepsilon_b$ 的综合结果就能保证高频子带的量化步长被推导得相对合理，且**无需额外传输参数**。同时尾数 μ_b 保持为 μ_0 不变，确保细调的平滑性。

4. 三种压缩模式对比

压缩模式	参数传递方式	μ_b 如何定	ε_b 如何定	Δ_b 与死区
无损压缩	无需传输量化参数	固定为 0	固定为 R_b	$\Delta_b = 1$ ，无死区（所有整数系数被精确保留）
有损（显式量化）	每个子带单独传输	编码器动态选择（0~2047）	编码器动态选择（0~31）	$\Delta_b = 2^{R_b - \varepsilon_b} (1 + \mu_b / 2^{11})$
有损（推导量化）	只传最深层 LL 的参数	所有子带 $\mu_b = \mu_0$	外推 $\varepsilon_b = \varepsilon_0 + n_b - N_L$	死区由外推得出的 Δ_b 决定，高频子带自动适配

核心一句话：无损压缩的参数是**"算出来"**的（硬性数学约束）；有损压缩的参数是**"选出来"**的（编码器率失真优化）。在有损压缩中，显式量化让编码器拥有最大自由度，推导量化则利用小波多级分解的增益规律，牺牲少许灵活性来大幅节省码流中量化参数的传输开销。

5. 视觉加权

JPEG 2000 允许对不同子带用不同权重（人眼对低频更敏感 → 低频步长更小），通过 **QCD** 标记的 **SPqcd** 值控制。

7. EBCOT Tier-1：位平面编码

EBCOT = Embedded Block Coding with Optimized Truncation（带优化截断的嵌入式块编码）

1. 位平面分解

每个 code-block 的量化系数按位平面从高到低逐层编码：

系数值（二进制）	位平面
13 = 0 1 1 0 1	bit4 (MSB) ← 先编码
7 = 0 0 1 1 1	bit3
-3 = 1 1 1 0 1	bit2
0 = 0 0 0 0 0	bit1
1 = 0 0 0 0 1	bit0 (LSB) ← 后编码

2. 三个编码通道（每个位平面依次执行）

通道	英文	编码对象
SPP	Significance Propagation Pass	有"显著邻居"但自身尚未显著的系数
MRP	Magnitude Refinement Pass	已经显著的系数（细化幅值）
CUP	Cleanup Pass	其余系数（含游程编码）

执行顺序：SPP → MRP → CUP，每个位平面重复。

3. 上下文建模

上下文类型	数量	依据
显著性上下文	19 个	8 邻域中已显著系数的分布
符号上下文	3 个	水平/垂直邻居的符号
幅值细化上下文	3 个	是否首次细化

4. MQ 算术编码器

- 基于 JBIG2 的 **MQ coder**;
- 二进制算术编码，自适应概率估计;
- 每个上下文维护一个状态（概率估计 + 最大概率符号 MPS）;
- 输出接近熵极限，比霍夫曼高效。

5. 可截断码流

每个编码通道结束后产生一个**截断点** (truncation point) ，记录：

- 该点的码率（累计字节数）
- 该点的失真（MSE 贡献）

这为 Tier-2 的率失真优化提供依据。

8. EBCOT Tier-2：分层打包

1. 率失真优化 (PCRD-opt)

PCRD = Post-Compression Rate-Distortion optimization

目标：在总码率约束下最小化总失真。

$$\min \sum_i D_i^{n_i} \quad \text{s.t.} \quad \sum_i R_i^{n_i} \leq R_{\text{budget}}$$

用拉格朗日乘子法求解：

$$\min \sum_i (D_i^{n_i} + \lambda R_i^{n_i})$$

对每个码块，选择使 $D_i + \lambda R_i$ 最小的截断点 n_i 。调整 λ 即可得到不同码率下的最优解。

几何解释：在率失真平面上找凸包 (convex hull) 上的点。



2. 质量层 (Layer)

- 每个 layer 对应一个目标码率/质量；
- 一个 layer 包含所有码块在该质量下的贡献；
- 解码端收到前 k 个 layer 就能重建出对应质量的图像。

3. 包 (Packet) 结构

Packet = 包头 + 包体

包头 (Packet Header) :

- └ 是否包含该码块的贡献 (1 bit/码块)
- └ 零位平面信息 (tag tree 编码)
- └ 编码通道数
- └ 码流长度增量

包体 (Packet Body) :

- └ 各码块的编码通道数据

Tag Tree: 一种层级编码结构, 高效表示"哪些码块在本层有贡献"。

9. 三重渐进性

JPEG 2000 码流天然支持三种渐进方式，且可**组合**：

渐进方式	机制	效果
质量渐进 (SNR)	按 layer 顺序传输	先模糊后清晰
分辨率渐进 (Resolution)	按 DWT 层级顺序传输	先小图后大图
空间渐进 (Spatial)	按 precinct 顺序传输	先局部后整体

渐进顺序由 `COD` 标记的 **progression order** 字段控制：

值	顺序	含义
0	LRCP	Layer → Resolution → Component → Position
1	RLCP	Resolution → Layer → Component → Position
2	RPCL	Resolution → Position → Component → Layer
3	PCRL	Position → Component → Resolution → Layer
4	CPRL	Component → Position → Resolution → Layer

10. 感兴趣区域 (ROI)

1. Maxshift 方法

把 ROI 区域的系数整体左移 s 位，使其位平面高于背景：

背景系数：

0 0 0 1 0 1 1 0 1 0 1

ROI 系数：

1 0 1 1 0 1 0 1 0 0 0

← 左移 s 位

↑

ROI 的高位平面先被编码

优点：不需要形状信息，解码端只需知道 s 值即可分离 ROI。

2. Scaling 方法

给 ROI 系数乘一个大于 1 的缩放因子，效果类似但需要传输 ROI 形状掩码。

11. 文件格式

1. 两种封装

格式	扩展名	说明
Codestream	.j2k / .j2c	裸码流，只有标记段
JP2	.jp2	带 box 结构的容器（推荐）
JPX	.jpx	扩展版，支持多帧、动画
MJ2	.mj2	Motion JPEG 2000 视频

2. JP2 Box 结构

长度（4B）	类型（4B）	数据（长度-8 字节）
--------	--------	-------------

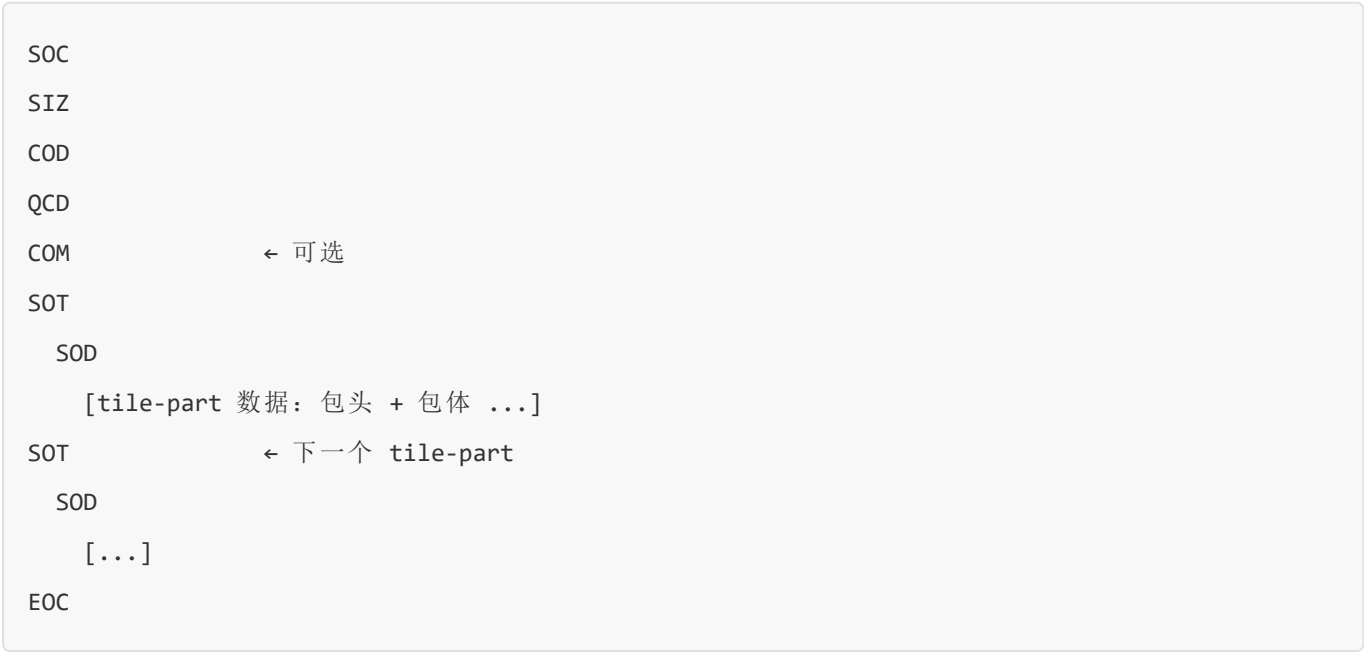
主要 Box：

Box	说明
jP (signature)	固定值 0x0D0A870A，标识 JP2 文件
ftyp	文件类型与兼容性
jp2h	图像头（超级 box）
└ ihdr	图像高度、宽度、分量数、位深、压缩类型
└ bpcc	位深不一致时的补充信息
└ colr	色彩空间（sRGB / sYCC / ICC）
└ pclr	调色板
└ cmap	分量映射
└ cdef	通道定义（哪个是 Alpha）
└ res	分辨率信息
jp2c	码流本体 （Contiguous Codestream）
jp2i	知识产权信息
xml	XML 元数据
uuid	厂商扩展
uinf / ulst / url	链接与引用

3. 码流标记段

标记	值	名称	说明
SOC	0xFF4F	Start of Codestream	码流开始
SIZ	0xFF51	Image and Tile Size	图像/tile 尺寸、分量、位深
COD	0xFF52	Coding Style Default	默认编码风格（小波级数、码块大小、渐进顺序）
COC	0xFF53	Coding Style Component	分量级覆盖
TLM	0xFF55	Tile-part Lengths	tile-part 长度索引
PLM	0xFF57	Packet Length Main	包头长度（主）
PLT	0xFF58	Packet Length Tile	包头长度（tile）
QCD	0xFF5C	Quantization Default	默认量化参数
QCC	0xFF5D	Quantization Component	分量级量化
RGN	0xFF5E	Region of Interest	ROI 定义
POC	0xFF5F	Progression Order Change	渐进顺序变更
PPM	0xFF60	Packed Packet Headers (Main)	打包包头（主）
PPT	0xFF61	Packed Packet Headers (Tile)	打包包头（tile）
CRG	0xFF63	Component Registration	分量配准
COM	0xFF64	Comment	注释
SOT	0xFF90	Start of Tile-part	tile-part 开始
SOP	0xFF91	Start of Packet	包开始（可选）
EPH	0xFF92	End of Packet Header	包头结束（可选）
SOD	0xFF93	Start of Data	tile-part 数据开始
EOC	0xFFD9	End of Codestream	码流结束

4. 典型码流布局



12. JPEG 2000 家族成员

标准	名称	说明
Part 1 (15444-1)	Core Coding System	基础，本文主要内容
Part 2 (15444-2)	Extensions	任意小波、多分量变换、非可逆色彩
Part 3 (15444-3)	Motion JPEG 2000	视频（MJ2 文件格式）
Part 4 (15444-4)	Conformance	一致性测试
Part 5 (15444-5)	Reference Software	参考实现（JJ2000 / OpenJPEG）
Part 6 (15444-6)	Compound Image	复合文档（文字+图像）
Part 8 (15444-8)	JP Secure	安全（加密、签名）
Part 9 (15444-9)	JPIP	交互式协议（渐进传输）
Part 10 (15444-10)	JP3D	三维（体数据）
Part 11 (15444-11)	JPWL	无线传输抗误码
Part 12 (15444-12)	ISO Base Media	与 MP4 共用的容器
Part 15 (15444-15)	HTJ2K	高吞吐量版，编码速度提升 10 倍以上

1. HTJ2K（High-Throughput JPEG 2000）

- 用 **FBCOT**（Fast Block Coding with Optimized Truncation）替代 EBCOT Tier-1；
- 用 **MEL + VLC** 替代 MQ 算术编码（可用 SIMD 加速）；
- 吞吐量提升 **10–30 倍**，压缩率仅损失约 2–5%；
- 已被 DCI 数字电影、广播行业采纳。

第三部分 对比与选型

1. JPEG vs JPEG 2000 全面对比

维度	JPEG	JPEG 2000
变换	8×8 DCT (分块)	多级 DWT (全局)
块效应	明显 (8×8 网格)	无 (小波无块结构)
熵编码	霍夫曼 (基线) / 算术 (扩展)	MQ 算术编码 (强制)
有损/无损	需切换模式 (SOF0 vs SOF3)	统一框架 (9/7 vs 5/3)
渐进传输	渐进模式 (有限)	三重渐进 (质量/分辨率/空间)
ROI	不支持	支持 (Maxshift)
压缩率 (同质量)	基准	提升 20–40%
压缩率 (无损)	约 2:1 (SOF3)	约 2:1–3:1 (5/3)
编码速度	快	慢 (约 3–10 倍)
解码速度	快	慢 (约 2–5 倍)
内存占用	低 (8×8 块)	高 (需缓存整个 tile)
专利	无	历史上部分技术有专利争议
浏览器支持	全部原生支持	Safari 原生; 其他需 JS 解码器
抗误码	重启间隔	码块独立 + JPWL
典型扩展名	.jpg .jpeg .jfif	.jp2 .j2k .jpx .jpf

2. 应用场景

1. JPEG 的主场

场景	原因
网页图片	浏览器原生支持、解码快
数码相机	硬件编码器成熟、速度快
社交媒体	生态完善、工具链齐全
视频帧内编码	Motion JPEG、DV
扫描文档	速度快、体积可接受

2. JPEG 2000 的主场

场景	原因
医学影像 (DICOM)	无损 + 有损统一、ROI、渐进
数字电影 (DCI)	250 Mbps 高码率、无块效应、可随机访问
档案保存	无损、开放标准、长期可读
遥感/地理	大图渐进、ROI、多分辨率
印刷出版	高质量、无块效应
JPIP 交互浏览	按需传输局部区域

3. 为什么 JPEG 2000 没有取代 JPEG

原因	说明
计算复杂度	编码慢 3-10 倍，早期硬件不支持
生态惯性	JPEG 工具链、硬件编解码器无处不在
浏览器缺失	除 Safari 外无原生支持，需 JS/WASM 解码器
专利顾虑	早期部分公司主张专利，阻碍采用
"够用就好"	网页场景 JPEG 质量已足够，WebP/AVIF 又提供了新选择
新竞争者	WebP、AVIF、HEIC、JPEG XL 分流了需求

4. 常见误区

误区	事实
"JPEG 质量 100 就是无损"	不是。DCT 浮点舍入 + 色度子采样仍有损
"JPEG 2000 只是更好的 JPEG"	架构完全不同：小波 vs DCT，算术 vs 霍夫曼
"JPEG 2000 一定比 JPEG 小"	低码率下优势明显；高码率下差距缩小
"JPEG 2000 无损压缩率很高"	约 2:1–3:1，不如 PNG 在图形上的表现
"渐进式 JPEG 就是 JPEG 2000"	完全不同。渐进式仍是 DCT + 霍夫曼
"JPEG 2000 没有专利问题"	Part 1 核心专利已过期，但历史上确实阻碍了采用
"小波变换没有伪影"	有振铃（ringing），但没有块效应
"JPEG 只能 8-bit"	扩展模式支持 12-bit；JPEG 2000 支持最高 38-bit

附录 速查表

A. JPEG 标记段速查

SOI	0xFFD8	文件开始
APP0	0xFFE0	JFIF
APP1	0xFFE1	EXIF / XMP
APP2	0xFFE2	ICC Profile
APP14	0xFFEE	Adobe
DQT	0xFFDB	量化表
DHT	0xFFC4	霍夫曼表
DAC	0xFFCC	算术编码条件
SOF0	0xFFC0	基线顺序
SOF1	0xFFC1	扩展顺序
SOF2	0xFFC2	渐进
SOF3	0xFFC3	无损
DRI	0xFFDD	重启间隔
SOS	0xFFDA	扫描开始
RSTn	0xFFD0	重启标记 (n=0..7)
COM	0xFFFE	注释
EOI	0xFFD9	文件结束

B. JPEG 2000 标记段速查

SOC	0xFF4F	码流开始
SIZ	0xFF51	图像/tile 尺寸
COD	0xFF52	编码风格（默认）
COC	0xFF53	编码风格（分量）
QCD	0xFF5C	量化（默认）
QCC	0xFF5D	量化（分量）
RGN	0xFF5E	ROI
POC	0xFF5F	渐进顺序变更
PPM	0xFF60	打包包头（主）
PPT	0xFF61	打包包头（tile）
COM	0xFF64	注释
SOT	0xFF90	tile-part 开始
SOP	0xFF91	包开始
EPH	0xFF92	包头结束
SOD	0xFF93	数据开始
EOC	0xFFD9	码流结束

C. 关键公式速查

项目	公式
FDCT	$F(u, v) = \frac{1}{4}C(u)C(v) \sum_{x,y} f(x, y) \cos \frac{(2x+1)u\pi}{16} \cos \frac{(2y+1)v\pi}{16}$
量化	$F_q = \text{round}(F/Q)$
反量化	$\hat{F} = F_q \cdot Q$
质量缩放	$S = 5000/Q \ (Q < 50) \ ; \ S = 200 - 2Q \ (Q \geq 50)$
RCT 正	$Y = \lfloor (R + 2G + B)/4 \rfloor, C_b = B - G, C_r = R - G$
RCT 逆	$G = Y - \lfloor (C_b + C_r)/4 \rfloor, R = C_r + G, B = C_b + G$
5/3 预测	$d[n] -= \lfloor (s[n] + s[n + 1])/2 \rfloor$
5/3 更新	$s[n] += \lfloor (d[n - 1] + d[n] + 2)/4 \rfloor$
9/7 缩放	$d *= -K, s /= K \ (K = 1.230174105)$
子带动态范围	$R_b = R_s + \log_2 G_b \ (G_b: \text{LL}=1, \text{HL/LH}=2, \text{HH}=4)$
量化步长 (JPEG 2000)	$\Delta_b = 2^{R_b - \varepsilon_b} \times (1 + \mu_b/2^{11})$
反量化 (JPEG 2000)	$\hat{c} = \text{sign}(q)(q + 0.5)\Delta_b \ (q \neq 0) \ ; \ q = 0 \Rightarrow \hat{c} = 0$
推导量化外推	$\mu_b = \mu_0, \ \varepsilon_b = \varepsilon_0 + n_b - N_L$
无损量化参数	$\mu_b = 0, \ \varepsilon_b = R_b, \ \Delta_b = 1$
PCRD	$\min \sum_i (D_i^{n_i} + \lambda R_i^{n_i})$

D. 参数默认值速查

参数	JPEG	JPEG 2000
块/变换尺寸	8×8	小波级数通常 5–6 级
码块大小	—	64×64（默认），可选 32×32 / 16×16
色度子采样	4:2:0（默认）	通常不采样（小波自带多分辨率）
质量因子	75（常见默认）	用压缩比或目标码率表示
渐进层数	扫描次数	通常 1–20 层
小波滤波器	—	9/7（有损） / 5/3（无损）

E. 术语中英对照

英文	中文
DCT (Discrete Cosine Transform)	离散余弦变换
DWT (Discrete Wavelet Transform)	离散小波变换
Quantization	量化
ZigZag Scan	之字形扫描
Huffman Coding	霍夫曼编码
Arithmetic Coding	算术编码
Run-Length Encoding (RLE)	游程编码
DPCM (Differential PCM)	差分脉冲编码调制
MCU (Minimum Coded Unit)	最小编码单元
Blocking Artifact	块效应
Ringing Artifact	振铃效应
Mosquito Noise	蚊噪
Lifting Scheme	提升方案
Subband	子带
Code-block	码块
Precinct	空间分区
Packet	包
Layer	层
Bit-plane	位平面
Truncation Point	截断点
Rate-Distortion Optimization	率失真优化
ROI (Region of Interest)	感兴趣区域
Progressive	渐进

英文	中文
Lossless / Lossy	无损 / 有损
Baseline	基线
Codestream	码流
Marker Segment	标记段
Byte Stuffing	字节填充
Deadzone Quantization	死区量化
Context Modeling	上下文建模
Tag Tree	标签树
Convex Hull	凸包
Symmetric Extension	对称延拓
Tile	分块 (JPEG 2000)
Component	分量
Subsampling	子采样
Luma / Chroma	亮度信号 / 色度信号
Luminance / Chrominance	亮度 / 色度 (物理量)
FWT (Fast Wavelet Transform)	快速小波变换
Lazy Wavelet	惰性小波 (提升方案的分裂步)
Vanishing Moments	消失矩
Expounded Quantization	显式量化
Derived Quantization	推导量化
Exponent / Mantissa	指数 / 尾数
NLLL Subband	最深层低频子带
RDO (Rate-Distortion Optimization)	率失真优化
Deadzone	死区
Tile-part	分块片段
Quality Layer	质量层

英文	中文
PCRD-opt	压缩后率失真优化
EBCOT	带优化截断的嵌入式块编码
MQ Coder	MQ 算术编码器
HTJ2K	高吞吐量 JPEG 2000
JPIP	JPEG 2000 交互式协议

F. JPEG 2000 量化参数速查

参数	含义	位宽	是否传输	取值范围
R_s	源图像精度	—	由 <code>SIZ.Ssiz</code> 隐含	8 / 12 / 16 ...
G_b	子带增益	—	由 <code>COD</code> 变换位隐含	LL=1, HL/LH=2, HH=4
R_b	理论动态范围	—	✗ 双方默认	$R_s + \log_2 G_b$
ε_b	粗调指数	5 位	✓	0 ~ 31
μ_b	细调尾数	11 位	✓	0 ~ 2047
Δ_b	实际量化步长	—	计算得到	$2^{R_b - \varepsilon_b} (1 + \mu_b / 2^{11})$
死区	归零区间	—	自动产生	$(-\Delta_b, \Delta_b)$, 宽度 $2\Delta_b$

三种模式对照：

模式	μ_b	ε_b	Δ_b
无损	0	R_b	1
有损 · 显式量化	每子带独立选择	每子带独立选择	每子带独立计算
有损 · 推导量化	全部 = μ_0	$\varepsilon_0 + n_b - N_L$	由外推值计算